

FICHE DE TD 2/3 • MOYEN

Informatique • Tle C D E

Écriture et exécution d'algorithmes intégrant une structure de données

Module 3 : Algorithmique et programmation

Cette fiche évalue la maîtrise des structures de données (tableaux, enregistrements) et des algorithmes classiques de recherche et de tri. Tout algorithme sera écrit en pseudo-code normalisé. Les tables d'exécution doivent être complètes et rigoureuses.

EXERCICE 1 Gestion des notes d'une série – Tableaux et Enregistrements

On souhaite traiter les notes de l'épreuve d'Informatique pour les 42 élèves de la série C du Lycée Bilingue de Nkongsamba. On définit le type `Eleve` (enregistrement) contenant : `Matricule` (chaîne), `Nom` (chaîne), `NoteInfo` (entier entre 0 et 20). On déclare le tableau `Promo` de 42 éléments de type `Eleve`, indicé de 1 à 42.

1. Écrire l'algorithme `SaisirPromo` qui remplit le tableau `Promo` en contrôlant que la note saisie est comprise entre 0 et 20 (boucle `TantQue` pour la saisie de la note).
2. Écrire l'algorithme `AfficherAdmis` qui parcourt `Promo` et affiche le matricule, le nom et la note des élèves ayant une note ≥ 10 . Si aucun élève n'est admis, afficher « Aucun admis ».
3. Écrire une fonction `MoyenneSerie(Promo)` qui calcule et renvoie la moyenne des notes de la série (réel).
4. Proposer une modification de la structure `Eleve` pour y ajouter la spécialité (chaîne : « Maths », « Physique », « Info »). Comment adapter l'algorithme de la question 2 pour n'afficher que les admis de la spécialité « Info » ?

EXERCICE 2 Recherche séquentielle – Stock de médicaments

La pharmacie « Santé Plus » à Yaoundé gère un stock de 150 références de médicaments. Chaque référence est stockée dans un tableau `Stock` indicé de 1 à 150, contenant des enregistrements de type `Medicament` : `Code` (entier), `Libelle` (chaîne), `Quantite` (entier), `PrixUnitaire` (réel). Les codes ne sont pas triés.

1. Écrire l'algorithme complet `RechercherMedoc(Stock, CodeCherche)` utilisant une recherche séquentielle avec un drapeau (booléen). L'algorithme renvoie l'indice de l'élément trouvé ou 0 si absent. Préciser la déclaration des variables locales.
2. On exécute cet algorithme pour `CodeCherche = 405`. Le tableau `Stock` contient aux indices 1 à 5 les codes : 102, 405, 301, 405, 210. Construire la table d'exécution complète (colonnes : Ligne, Instruction, Variables modifiées, Drapeau, Indice, Écran) jus-

qu'à l'arrêt de la boucle. Combien de comparaisons ont été effectuées?

- Justifier en une phrase pourquoi la recherche séquentielle est ici la seule méthode de recherche standard applicable directement sur ce tableau.

EXERCICE 3 Tri par insertion – Production cacaoyère

Une coopérative du Sud-Ouest enregistre la production annuelle (en kg) de 10 planteurs dans un tableau `Prod` indicé de 1 à 10. État initial :

Indice i	1	2	3	4	5	6	7	8	9	10
<code>Prod[i]</code>	1200	850	1500	900	1100	950	1300	800	1000	1400

On applique l'algorithme de tri par insertion croissante standard (boucle externe i de 2 à 10, clé = `Prod[i]`, décalage vers la droite tant que $j > 0$ et `Prod[j] > clé`).

- Donner l'état du tableau `Prod` après l'itération $i = 3$ (insertion de 1500) et après l'itération $i = 4$ (insertion de 900). Présenter sous forme de deux tableaux.
- Construire la table d'exécution détaillée pour l'itération $i = 6$ uniquement (insertion de la valeur 950). Colonnes : i, j , clé, `Prod[j]`, Condition ($j > 0$ ET `Prod[j] > clé`), Action (Affectation/Décalage), État partiel du tableau (indices 1 à 6).
- Quelle est la complexité en temps dans le pire des cas pour cet algorithme sur n éléments? Donner un exemple de configuration initiale de `Prod` (10 valeurs) qui réalise ce pire cas.

EXERCICE 4 Problème synthétique – Concours d'entrée

L'Université de Ngaoundéré organise un concours. Les 200 candidats sont stockés dans un tableau `Candidats` (enregistrements : `NumInscri`, `Nom`, `NoteMaths`, `NotePhys`, `Moyenne`). La moyenne est déjà calculée. Le tableau n'est pas trié.

- Écrire l'algorithme `ClasserMerite(Candidats)` qui trie le tableau par ordre décroissant de `Moyenne` en utilisant le tri par insertion (adapter la condition de comparaison).
- Après tri, on veut déterminer le rang d'un candidat dont on connaît le `NumInscri`. Écrire l'algorithme `ChercherRang(Candidats, Num)` qui utilise une recherche séquentielle (le tableau est maintenant trié, mais on impose la séquentielle). L'algorithme affiche : « Candidat [`Nom`] classé au rang [`Rang`] » ou « Introuvable ».
- On décide d'admettre les 50 premiers. Écrire l'algorithme `GenererListeAdmis(Candidats, Admis)` qui recopie les 50 premiers enregistrements du tableau trié `Candidats` vers un nouveau tableau `Admis` (taille 50).

4. Si deux candidats ont exactement la même moyenne, l'ordre relatif entre eux après le tri par insertion dépend de leur ordre initial. Quel nom porte cette propriété de tri ? Justifier brièvement.

EXERCICE 5 Analyse d'une table d'exécution – Algorithme mystère

On donne l'algorithme suivant opérant sur un tableau $T[1..N]$ d'entiers :

Algorithme Mystere

Variables T : Tableau[1..N] d'entiers ; i, j, temp : Entier

Début

Pour i de 2 à N faire

temp $\leftarrow T[i]$; $j \leftarrow i - 1$

TantQue ($j \geq 1$) ET ($T[j] > \text{temp}$) faire

$T[j+1] \leftarrow T[j]$; $j \leftarrow j - 1$

FinTantQue

$T[j+1] \leftarrow \text{temp}$

FinPour

Fin

On exécute cet algorithme avec $N = 5$ et l'état initial : $T = [22, 11, 33, 5, 44]$.

1. Identifier formellement l'algorithme représenté.
2. Compléter la table d'exécution ci-dessous pour les itérations $i = 2$ et $i = 3$ uniquement.

i	j	temp	Condition TantQue	Affectation $T[j+1]$	État $T[1..i]$	Écran
2	1	11	VRAI (11>22)	$T[2] \leftarrow 22$	[22, 22, ...]	–
2	0	11	FAUX	$T[1] \leftarrow 11$	?	–
3	?	?	?	?	?	–

3. Donner l'état final du tableau T après l'exécution complète ($i = 2$ à 5).
4. Si on remplace la condition $T[j] > \text{temp}$ par $T[j] < \text{temp}$, quel devient l'ordre de tri final ? Donner l'état final pour le même jeu de données initial.